

第 7 章 图论基础

图由顶点集合与边集合构成，记作

$G = (V, E)$ 。若边没有方向称为无向图，有方向则称为有向图。

无向图中顶点的度是与该顶点相关联的边的数目。有向图中分为入度与出度，入度是以该顶点为终点的边数，出度是以该顶点为起点的边数。

握手定理指出，无向图中所有顶点的度数之和等于边数的两倍。由此可知，任何图中度数为奇数的顶点个数必为偶数。

路径是顶点与边交替出现的序列，其中顶点不重复的路径称为简单路径。起点与终点重合的简单路径称为简单回路。

连通性是无向图的重要性质：若任意两个顶点之间都存在路径，则称该图为连通图，否则称为非连通图，其极大连通子图称为连通分量。

生成树是连通图的极小连通子图，它包含图中全部顶点且只含 $n - 1$ 条边。带权图的生成树中权值之和最小的称为最小生成树。

求最小生成树的经典算法有两种：普里姆算法从任一顶点出发逐步扩展，时间复杂度为 $O(n^2)$ ，适合稠密图；克鲁斯卡尔算法按边权递增选边并用并查集判环，时间复杂度为 $O(e \log_2 e)$ ，适合稀疏图。

最短路径问题中，迪杰斯特拉算法按路径长度递增的次序产生最短路径，适用于边权非负的情形；弗洛伊德算法通过三重循环递推任意两点间的最短距离，时间复杂度为 $O(n^3)$ 。

拓扑排序是对有向无环图的顶点排成线性序列，使每条边都从前驱指向后继。若图中存在回路，则拓扑排序无法完成，因此可用它判断工程是否可顺利推进。

关键路径是活动网络（AOE

网）中从源点到汇点路径长度最大的路径，其上的活动称为关键活动，缩短非关键活动并不能缩短整个工期。

下图为本节相关示意图：

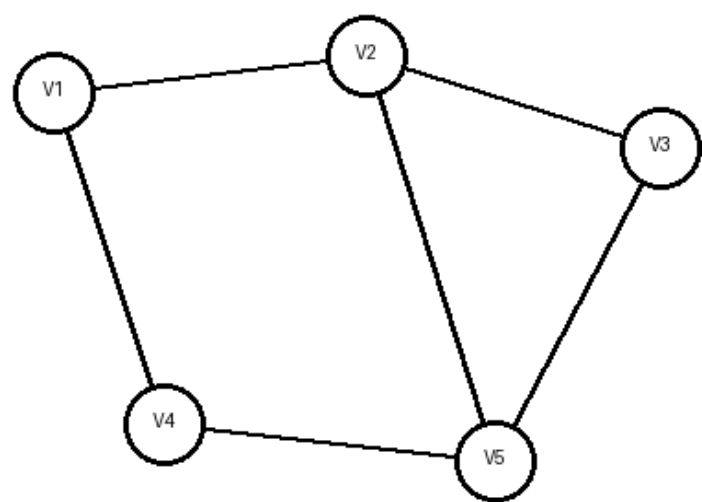


Figure: a weighted graph (synthetic test fixture)

第 8 章 图的存储与遍历

图的存储方式主要有邻接矩阵与邻接表两种。邻接矩阵用 n 阶方阵表示顶点之间的相邻关系，适合稠密图，判断两点是否相邻的时间复杂度为 $O(1)$ 。

邻接表为每个顶点建立一条单链表，存储其所有邻接顶点，适合稀疏图，占用的存储空间与顶点数和边数之和成正比。

深度优先搜索沿一条路径尽可能深入，直到无法继续再回溯，通常借助递归或显式栈实现，其时间复杂度在邻接表下为 $O(n + e)$ 。

广度优先搜索按层次逐层扩展，借助队列实现，可以求出无权图中从起点到其余顶点的最短路径长度。

对同一个图，深度优先搜索得到的遍历序列不唯一，它取决于邻接表中顶点的存放次序以及起点的选择。

深度优先生成树与广度优先生成树分别由两种遍历过程中经过的边构成，它们都是原图的生成树（对连通图而言）。

利用深度优先搜索可以在 $O(n + e)$ 时间内判断有向图中是否存在回路：若在搜索过程中遇到仍在栈中的顶点，则存在回路。

第9章 查找

顺序查找从表的一端开始逐个比较，平均查找长度为 $(n + 1) / 2$ ，它对表的存储结构没有要求，适用于线性链表。

折半查找要求表有序且顺序存储，每次比较后把查找区间缩小一半，平均查找长度约为 $\log_2 n$ ，效率明显优于顺序查找。

分块查找把表分成若干块，块间有序、块内无序，先用索引表确定所在块再在块内顺序查找，是折半与顺序查找的折中方案。

二叉排序树的中序遍历序列是递增有序的，插入与删除都需保持这一性质，其查找效率与树的形态有关。

平衡二叉树要求任意结点的左右子树高度差的绝对值不超过 1，通过旋转操作在插入或删除后维持平衡，使查找的时间复杂度稳定在 $O(\log_2 n)$ 。

散列表通过散列函数把关键字映射到存储位置，理想情况下查找的时间复杂度为 $O(1)$ 。冲突不可避免，常用开放定址法与链地址法解决。

装填因子是散列表中已存记录数与表长之比，它越大冲突越多，平均查找长度也越长。
下图为本节相关示意图：

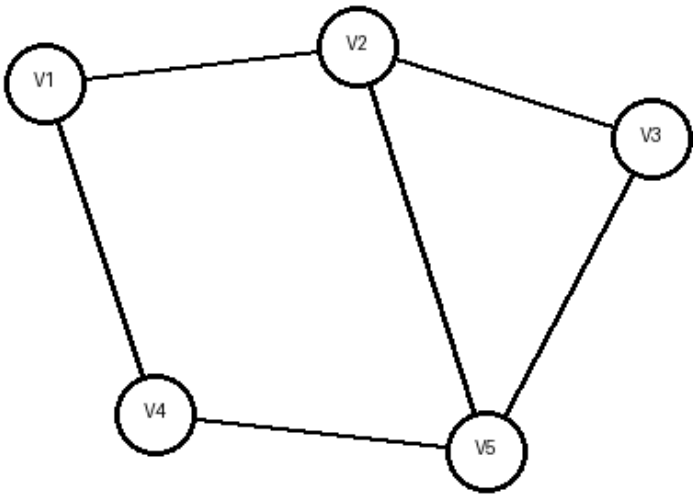


Figure: a weighted graph (synthetic test fixture)

第 10 章 排序

插入排序每次把一个记录插入到已排好序的子序列中，平均时间复杂度为 $O(n^2)$ ，当原始序列基本有序时效率较高，且是稳定的排序方法。

希尔排序是插入排序的改进，通过增量分组使记录大步移动，逐步减小增量至 1，平均时间复杂度约为 $O(n^{1.3})$ ，但稳定性被破坏。

冒泡排序相邻比较并交换，每趟把最大元素移到末尾，时间复杂度为 $O(n^2)$ ，是稳定排序。

快速排序选取枢轴元素进行划分，平均时间复杂度为 $O(n \log_2 n)$ ，最坏情况退化为 $O(n^2)$ ，是不稳定排序，通常被认为是内部排序中平均性能最好的方法。

简单选择排序每趟从未排序部分选出最小元素放到已排序末尾，比较次数与初始序列无关，总为 $n(n-1)/2$ ，是不稳定排序。

堆排序利用完全二叉树的性质建堆与调整，时间复杂度稳定为 $O(n \log_2 n)$ ，只用到常数级辅助空间，适合记录数很大的场合。

归并排序把序列不断二分再合并，时间复杂度为 $O(n \log_2 n)$ ，需要 $O(n)$ 的辅助空间，是稳定排序，适合外排序。

基数排序按关键字各位依次分配与收集，属于分配类排序，时间复杂度为 $O(d(n+r))$ ，是稳定排序。

第 1 1 章 树与二叉树

树是一种重要的非线性结构，它有且仅有一个根结点，其余结点分成若干互不相交的子树，树中结点的度是该结点拥有的子树个数。

二叉树是每个结点至多有两棵子树的树，且子树有左右之分、次序不能颠倒，这与度为 2 的树并不等价。

二叉树的第 i 层至多有 2 的 $i - 1$ 次方个结点，深度为 k 的二叉树至多有 2 的 k 次方减一个结点。

完全二叉树中除最后一层外其余各层都是满的，且最后一层的结点集中在左侧，因此可用顺序存储而不浪费空间。

对任何一棵二叉树，叶子结点数等于度为 2 的结点数加 1，这一性质在计算结点数量时经常用到。

二叉树的先序遍历按根、左、右的次序访问结点，中序遍历按左、根、右的次序访问，后序遍历按左、右、根的次序访问。

由先序序列与中序序列可以唯一确定一棵二叉树，由后序序列与中序序列同样可以唯一确定，但仅有先序与后序无法确定。

线索二叉树利用空指针域存放前驱与后继的线索，从而在遍历时不需要栈，节省了辅助空间。

哈夫曼树是带权路径长度最小的二叉树，构造时每次选取权值最小的两棵树合并，其带权路径长度可用于设计最优编码。

并查集用双亲表示法存储集合，通过路径压缩与按秩合并把查找的均摊时间复杂度降到接近常数级。

下图为本节相关示意图：

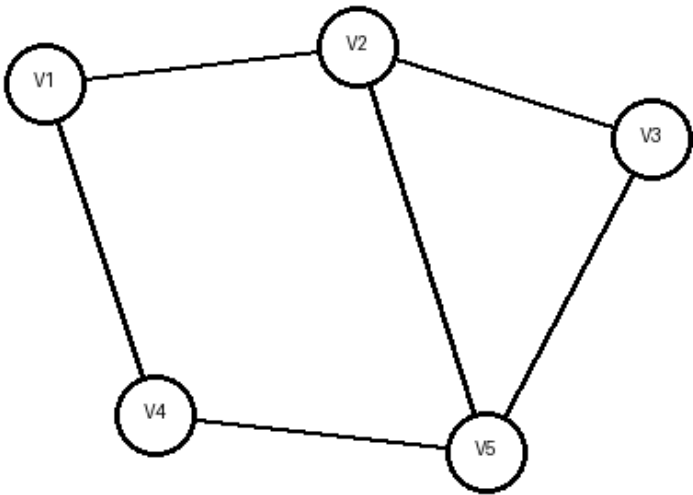


Figure: a weighted graph (synthetic test fixture)

第 1 2 章 线性表

线性表是由零个或多个数据元素组成的有限序列，除首尾元素外每个元素都有唯一的前驱与后继。

顺序表用一段连续存储单元依次存放元素，可以随机访问，按下标存取的时间复杂度为 $O(1)$ ，但插入与删除平均要移动一半元素。

链表用一组任意的存储单元存放元素，通过指针表示逻辑关系，插入与删除只需修改指针，但查找必须从头逐个比较。

单链表只能从头结点出发顺链查找，若要频繁访问前驱，可使用双向链表，或用带尾指针的循环链表简化尾端操作。

静态链表借助数组模拟指针，适合不支持动态分配的语言，其空链域的利用与数组下标密切相关。

在单链表中设置头结点可以统一空表与非空表的处理，简化插入与删除的边界判断，这是常见的工程写法。

顺序表与链表的选择取决于操作类型：查找为主且元素个数可预估时用顺序表，频繁插删且规模变化大时用链表。

合并两个有序线性表时，若用顺序表实现需要额外空间，若用链表实现可以原地修改指针，空间复杂度为 $O(1)$ 。

线性表的长度是元素个数，它是动态变化的，而数组的容量是预先分配的，二者必须区分清楚。

在顺序表中插入元素时平均移动 $n/2$ 个元素，删除时同样平均移动约 $n/2$ 个元素，这是其效率低于链表的主要原因。

第 1 3 章 栈与队列

栈是只允许在一端进行插入与删除的线性表，其特点是后进先出，允许操作的一端称为栈顶，另一端称为栈底。

顺序栈用数组实现，需要设置栈顶指针，入栈前必须判断上溢，出栈前必须判断下溢。

共享栈让两个栈共用一段存储空间、从两端向中间增长，从而提高空间利用率，但当两个栈顶相遇时即发生上溢。

链栈用单链表实现，栈顶设在链头，入栈与出栈都在链头进行，通常不需要头结点。

栈的典型应用包括括号匹配、表达式求值、以及把递归算法改写成非递归形式。

中缀表达式转后缀表达式时，操作数直接输出，运算符按优先级入栈或弹出，最终得到不含括号的后缀表达式。

队列是只允许在一端插入、另一端删除的线性表，其特点是先进先出，插入端称为队尾，删除端称为队头。

顺序队列会出现假溢出，循环队列通过对下标取模解决这一问题，并常牺牲一个单元来区分队空与队满。

链队列用单链表加队头与队尾指针实现，出队时需注意队列变空后要把队尾指针置空，否则会留下悬空引用。

双端队列允许两端都进行插入与删除，输入受限或输出受限的双端队列可用来判断给定的输出序列是否合法。

下图为本节相关示意图：

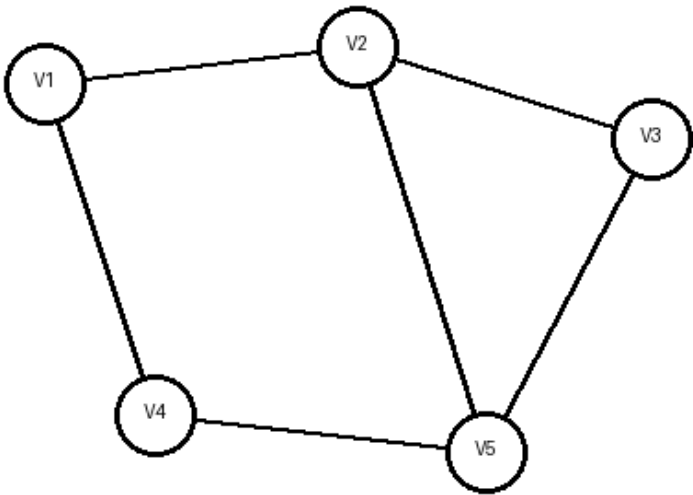


Figure: a weighted graph (synthetic test fixture)

第 14 章 串与数组

串是由零个或多个字符组成的有限序列，其中子串是串中任意个连续字符组成的子序列，空串不包含任何字符。

朴素的模式匹配算法在主串长度为 n 、模式串长度为 m 时最坏时间复杂度为 $O(nm)$ ，当模式串中有大量重复字符时效率很低。

KMP 算法预先计算模式串的 `next` 数组，利用已经比较过的信息避免主串指针回溯，把时间复杂度降到 $O(n+m)$ 。

`next` 数组的含义是模式串当前位置之前的子串中最长相等前后缀的长度，求解过程本身也是对模式串的自我匹配。

数组按下标访问的时间复杂度为 $O(1)$ ，其元素地址可由首地址与下标线性计算得出，因此多维数组要注意行优先还是列优先。

特殊矩阵如对称矩阵、三角矩阵可以用压缩存储，把 n 阶矩阵的元素个数从 n^2 减少到约 $n(n+1)/2$ 个。

稀疏矩阵用三元组表或十字链表存储非零元素，前者适合按行处理，后者适合频繁插入与删除。

广义表是线性表的推广，其元素可以是原子也可以是子表，表的深度指括号嵌套的最大层数，表长指最外层元素个数。

在字符串处理中，朴素的字符比较可以借助库函数完成，但理解 `next` 数组的推导过程仍是考察掌握程度的重要方式。

字典树把公共前缀合并到同一条路径上，适合大量字符串的检索与前缀查询，其空间开销与字符集大小有关。

第 15 章 时间复杂度与算法分析

算法的时间复杂度是问题规模 n 的函数，通常只保留最高阶项并忽略常数因子，用大 O 记号表示其渐近上界。

常见的时间复杂度由低到高依次为常数阶、对数阶、线性阶、线性对数阶、平方阶、立方阶与指数阶，其中指数阶算法在大规模输入下不可接受。

空间复杂度衡量算法运行过程中临时占用的存储空间，递归算法的空间开销主要来自递归工作栈的深度。

同一算法在最好、最坏与平均情况下的复杂度可能不同，评价时通常给出最坏情况以保证下界，同时关注平均情况以反映实际表现。

分治法把问题分解为若干规模更小的子问题，分别求解后合并结果，归并排序与快速排序都是分治思想的典型应用。

动态规划要求问题具有最优子结构与重叠子问题的性质，通过保存中间结果避免重复计算，其核心是写出状态转移方程。

贪心算法每一步都做出当前看起来最优的选择，它不保证全局最优，只有当问题满足贪心选择性质时才有效。

回溯法按深度优先的方式搜索解空间，遇到不满足约束的分支就剪枝回退，适合求解组合数较大的可行解问题。

在工程实践中，先用大 O 判断算法量级是否可行，再用实际测量验证常数因素，二者不可互相替代。

分析算法时要区分基本操作与辅助操作，基本操作的执行次数才决定复杂度，而输入规模的度量方式必须与算法本身匹配。

下图为本节相关示意图：

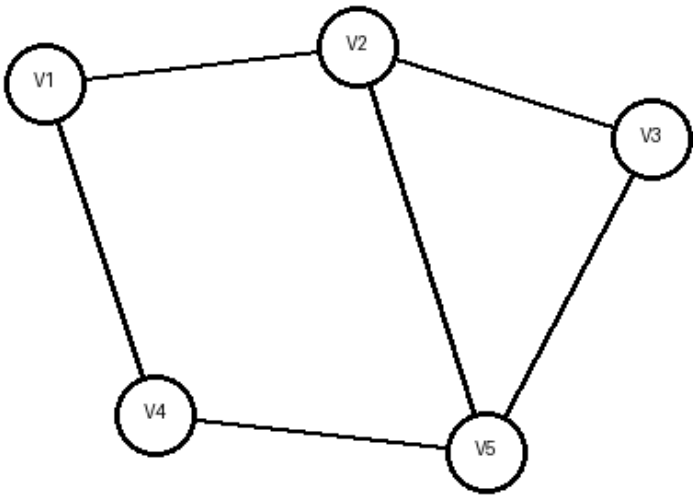


Figure: a weighted graph (synthetic test fixture)

第 16 章 文件与外排序

文件是存放在外部存储器上的记录集合，顺序文件适合批量处理，索引文件与散列文件则更适合随机访问。

外排序要先分段读入内存进行内部排序，把有序段写回磁盘，再通过多路归并把有序段合并成完整的有序文件。

多路平衡归并通过增大归并路数减少读写磁盘的趟数，但路数增大后内部比较的次数也会增加，需要用败者树来降低比较开销。

置换选择排序利用堆的性质在内存中生成尽可能长的初始归并段，从而减少初始段的个数。

最佳归并树按照哈夫曼树的思想构造，使归并过程中读写的记录总数最少。

在磁盘读写代价远高于内存比较代价的前提下，外排序优化的首要目标是减少磁盘访问次数，而不是减少比较次数。

因为外存访问以块为单位，所以缓冲区的大小与数量直接影响外排序的实际效率。

索引顺序文件兼顾顺序访问与随机访问，其索引项把文件分成若干块，查找时先查索引再在块内顺序查找。

在多路归并中，每个归并段只需要一个缓冲区，缓冲区数量与归并路数一致，因此内存容量限制了归并路数。

外排序的稳定性取决于内部排序阶段是否稳定，以及归并时相同关键字的处理次序。

第 1 7 章 算法设计策略

分治法适用于可以分解为相互独立子问题的情况，若子问题相互重叠就应该改用动态规划以避免重复计算。

动态规划既可以自底向上递推，也可以自顶向下记忆化搜索，前者节省递归开销，后者只计算真正用到的状态。

贪心算法的正确性必须证明贪心选择性质与最优子结构，仅凭直觉选择局部最优往往会得到反例。

回溯法的时间复杂度通常与解空间树的大小有关，剪枝函数越好，搜索的状态就越少。

分支限界法按广度优先或最小耗费优先搜索解空间，适合求解最优化问题，与回溯法的搜索次序不同。

在解决同一个问题时，选择哪种策略取决于问题的结构，而不是算法的知名度。

随机化算法以一定概率保证正确性或期望效率，在判定质数与最小割等问题上有广泛应用。

近似算法用于处理 NP 难问题，它不追求最优解，而是给出与最优解的比值有界的结果。

算法设计中要同时考虑正确性、可读性与效率，只追求效率的代码往往难以维护，也难以证明其正确。

把复杂问题拆成已经解决过的子问题，是算法设计中最常见也最有效的思路。

下图为本节相关示意图：

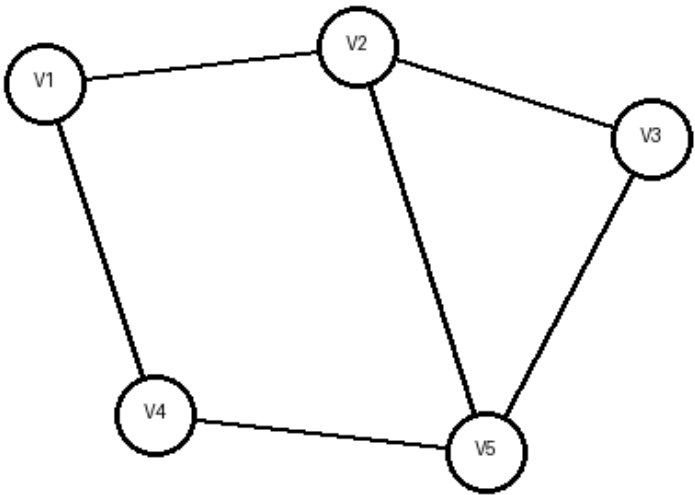


Figure: a weighted graph (synthetic test fixture)

第 18 章 数据结构在工程中的应用

哈希表用于常数时间的键值查找，缓存、去重与计数场景都依赖它，冲突处理策略直接影响其在真实数据下的表现。

有序集合常用平衡树或跳表实现，前者保证严格的对数上界，后者实现更简单且并发性能更好。

优先队列用堆实现，任务调度、定时器与 Top - K 统计都建立在它之上。

布隆过滤器用很小的空间判断元素是否可能存在，它的假阳性可以被容忍，但假阴性绝不能出现。

并查集在连通性判断、图的最小生成树与前缀合并等场景中都有应用，路径压缩是其实用性的关键。

字典树适合前缀检索与自动补全，其空间开销可以通过压缩单分支路径来降低。

环形缓冲区广泛用于生产者消费者模型与流式处理，它避免了频繁的内存分配。

位图用位来标记状态，在标记大规模互斥状态时比布尔数组节省大量空间。

在真实系统中，数据结构的选择往往决定了接口的复杂度，而不是反过来。

衡量一个数据结构是否合适，要看主要操作的时间复杂度、内存占用以及实现与维护的成本。